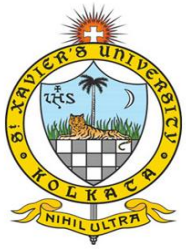


Professor: SVC Xavier Business School Dipayan Saha

Subject: Multi-Variate Analysis Sem: III, Roll: 0071

Specialization: Business Analytics

Assignment-II (imports_85)

I have assigned with the "imports_85" dataset to work on and impute the missing values in it and prepare a Regression Model to Predict the Car Price.

Here I'm describing the whole step by step working procedure.

I have attached the dataset in R and stored the name of the dataset as

b1= imports_85

Before Proceeding we need to install some libraries to work on:

library(mice)

library(VIM)

library(Hmisc)

Step 1: Checking missing values:

R Code: is.na(b1)

which(is.na(b1))

The output will show the positions of the missing values column wise. If missing value present, then 'TRUE' else 'FALSE'.

Step 2: Summary of the missing values of the Dataset:

R Code: summary(is.na(b1))

sum(is.na(b1)

Output: The output result will show the missing Values in the dataset for each variable and column and also the total number of missing values present in the dataset.

Total 59 missing values are present in the dataset.

```
> #Checking the Summary of the Missing Values
> summary(is.na(b1))
 symboling       normalized-losses    make          fuel-type       aspiration
 Mode :logical   Mode :logical     Mode :logical   Mode :logical   Mode :logical
 FALSE:205       FALSE:164         FALSE:205       FALSE:205       FALSE:205
                 TRUE :41
 num-of-doors    body-style      drive-wheels    engine-location wheel-base
 Mode :logical   Mode :logical   Mode :logical   Mode :logical   Mode :logical
 FALSE:203       FALSE:205       FALSE:205       FALSE:205       FALSE:205
 TRUE :2
   length          width           height        curb-weight     engine-type
 Mode :logical   Mode :logical   Mode :logical   Mode :logical   Mode :logical
 FALSE:205       FALSE:205       FALSE:205       FALSE:205       FALSE:205

 num-of-cylinders engine-size     fuel-system       bore            stroke
 Mode :logical    Mode :logical   Mode :logical   Mode :logical   Mode :logical
 FALSE:205        FALSE:205       FALSE:205       FALSE:201       FALSE:201
                                                  TRUE :4         TRUE :4
 compression-ratio horsepower       peak-rpm        city-mpg       highway-mpg
 Mode :logical     Mode :logical   Mode :logical   Mode :logical   Mode :logical
 FALSE:205         FALSE:203       FALSE:203       FALSE:205       FALSE:205
                   TRUE :2         TRUE :2
   price
 Mode :logical
 FALSE:201
 TRUE :4
> sum(is.na(b1))
[1] 59
```

Step 3: Checking Summary of the dataset:

R Code: summary(b1)

Output: The Output will show the summary of the whole dataset.

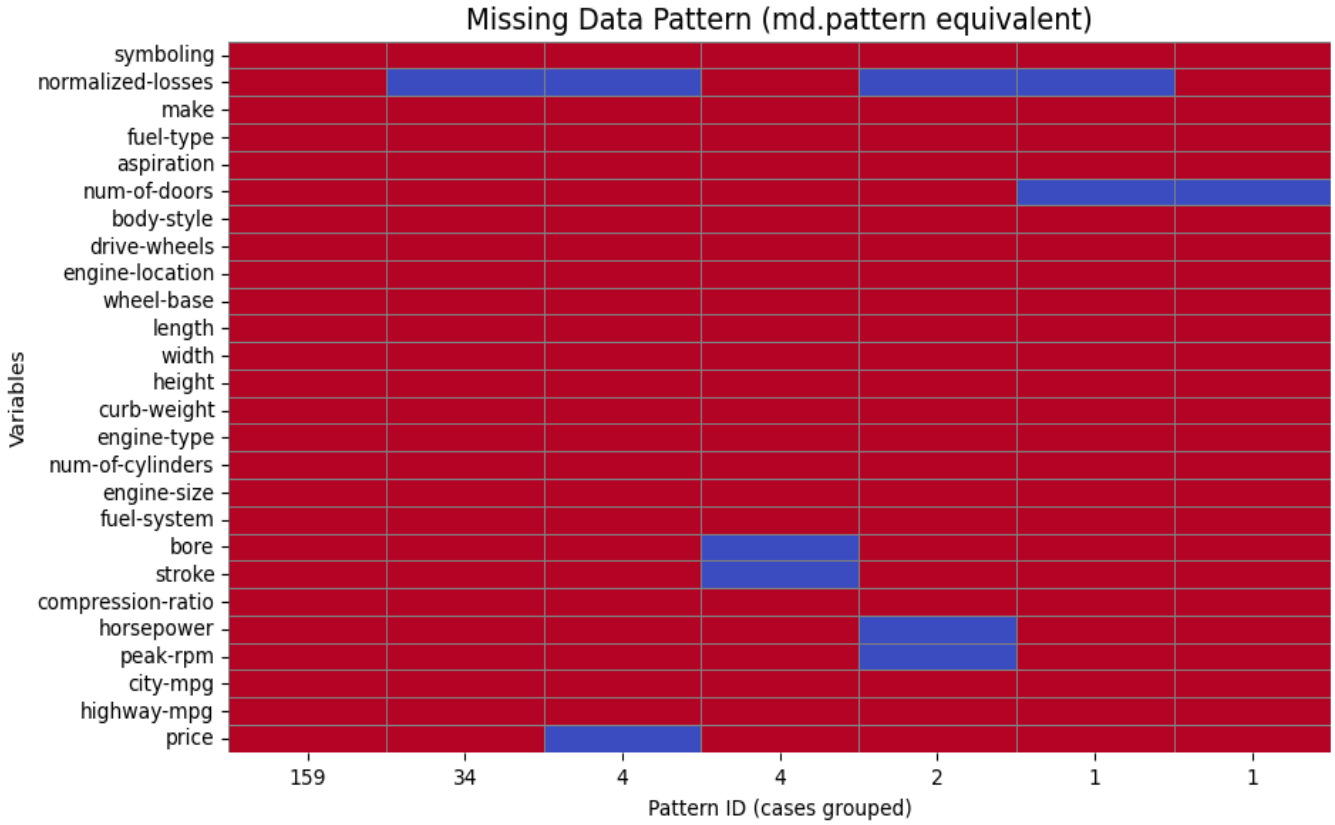
```
> t(summary(b1))

  symboling       Min.   :-2.0000   1st Qu.: 0.0000   Median : 1.0000   Mean   : 0.8341
normalized-losses Min.   : 65       1st Qu.: 94       Median :115       Mean   :122
    make          Length:205        Class :character  Mode  :character
 fuel-type        Length:205        Class :character  Mode  :character
 aspiration       Length:205        Class :character  Mode  :character
num-of-doors      Length:205        Class :character  Mode  :character
 body-style       Length:205        Class :character  Mode  :character
drive-wheels      Length:205        Class :character  Mode  :character
engine-location   Length:205        Class :character  Mode  :character
  wheel-base      Min.   : 86.60    1st Qu.: 94.50    Median : 97.00    Mean   : 98.76
    length        Min.   :141.1     1st Qu.:166.3     Median :173.2     Mean   :174.0
    width         Min.   :60.30     1st Qu.:64.10     Median :65.50     Mean   :65.91
    height        Min.   :47.80     1st Qu.:52.00     Median :54.10     Mean   :53.72
 curb-weight      Min.   :1488      1st Qu.:2145      Median :2414      Mean   :2556
engine-type       Length:205        Class :character  Mode  :character
num-of-cylinders  Min.   : 2.00     1st Qu.: 4.00     Median : 4.00     Mean   : 4.38
 engine-size      Min.   : 61.0     1st Qu.: 97.0     Median :120.0     Mean   :126.9
fuel-system       Length:205        Class :character  Mode  :character
     bore         Min.   :2.54      1st Qu.:3.15      Median :3.31      Mean   :3.33
    stroke        Min.   :2.070     1st Qu.:3.110     Median :3.290     Mean   :3.255
compression-ratio Min.   : 7.00     1st Qu.: 8.60     Median : 9.00     Mean   :10.14
  horsepower      Min.   : 48.0     1st Qu.: 70.0     Median : 95.0     Mean   :104.3
   peak-rpm       Min.   :4150      1st Qu.:4800      Median :5200      Mean   :5125
   city-mpg       Min.   :13.00     1st Qu.:19.00     Median :24.00     Mean   :25.22
 highway-mpg      Min.   :16.00     1st Qu.:25.00     Median :30.00     Mean   :30.75
    price         Min.   : 5118     1st Qu.: 7775     Median :10295     Mean   :13207
```

Step 4: Checking Missing data pattern:

R Code: t(md.pattern(b1))

But we are doing this pattern in python.

Output: The output will represent if there are any patterns within the missing dataset.



Step 5: Delete those rows/tuples with missing values in target/dependent variable:

We have to delete those tuples which have missing values in the target variable. Otherwise, these will impact our model. Our model should not consider those values.

R Code: which(is.na(b1$price)) #To identify the tuples

b2<-b1[-c(10,45,46,130),] #To Delete those tuples

View(b2) #To view the modified dataset

Step 6: To identify which variable have how many missing values:

R Code: colSums(is.na(b2))

The output will show that how many missing values are present in the remaining variables in the dataset.

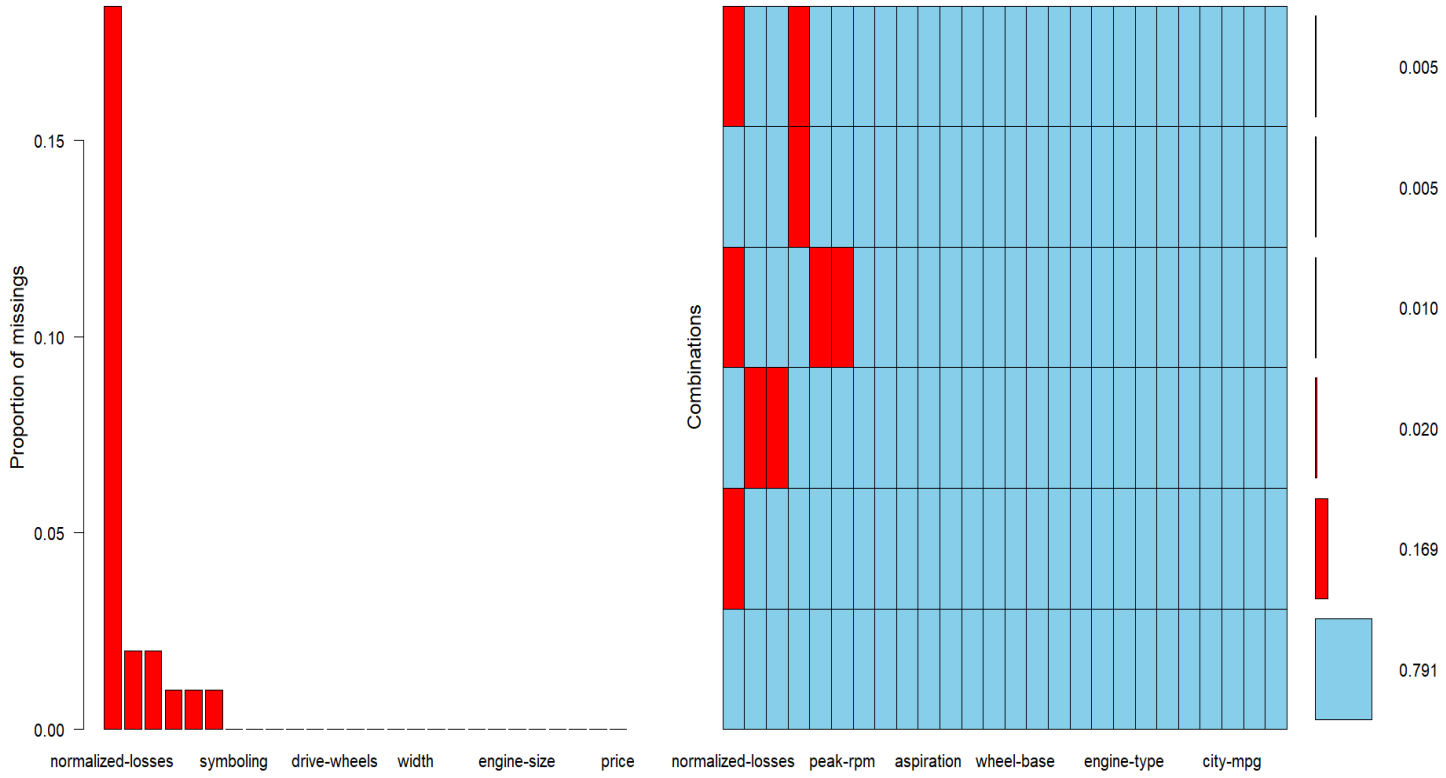
```
> colSums(is.na(b2))
       symboling normalized-losses             make        fuel-type       aspiration      num-of-doors
               0                37                0                0                0                2
      body-style      drive-wheels   engine-location       wheel-base           length            width
               0                0                0                0                0                0
          height       curb-weight       engine-type num-of-cylinders      engine-size      fuel-system
               0                0                0                0                0                0
            bore           stroke compression-ratio       horsepower         peak-rpm         city-mpg
               4                4                0                2                2                0
     highway-mpg            price
               0                0
```

Clearly it is visible, Normalized losses have so many missing values compare to other variables. Also, Bore, Stroke compression-ratio, horsepower and peak-rpm has a few missing values. We have to analyze the missing trend of these values and impute them before building our model.

Step 7: Using aggr() plot to visualize the pattern and amount of missing data in the dataset.

R Code: aggr(b2,pos=1,numbers=T,las=1,sortVars=T, labels=names(b2))

Output: The missingness of the dataset and the pattern of the missingness in the variables will be visible.



Step 8: Saving the modified dataset in the name of "Import_New".

R Code: write.csv(b2, "C:/Users/Biswaditya Saha/OneDrive/Desktop/XBS-MBA-BA-3rd SEM/Multi Variate Analysis-Shuvendu Sir/R Studio/Import_New.csv", row.names = FALSE)

The New Dataset will be saved in the local folder.

Step 9: Checking the type and pattern of the missing value using Parallel box plots to identify the imputation method.

Python Code: independent_vars = ['normalized-losses', 'num-of-doors', 'bore', 'stroke', 'compression-ratio', 'horsepower', 'peak-rpm']

for col in independent_vars:

    missing_col_name = f'{col}_missing'

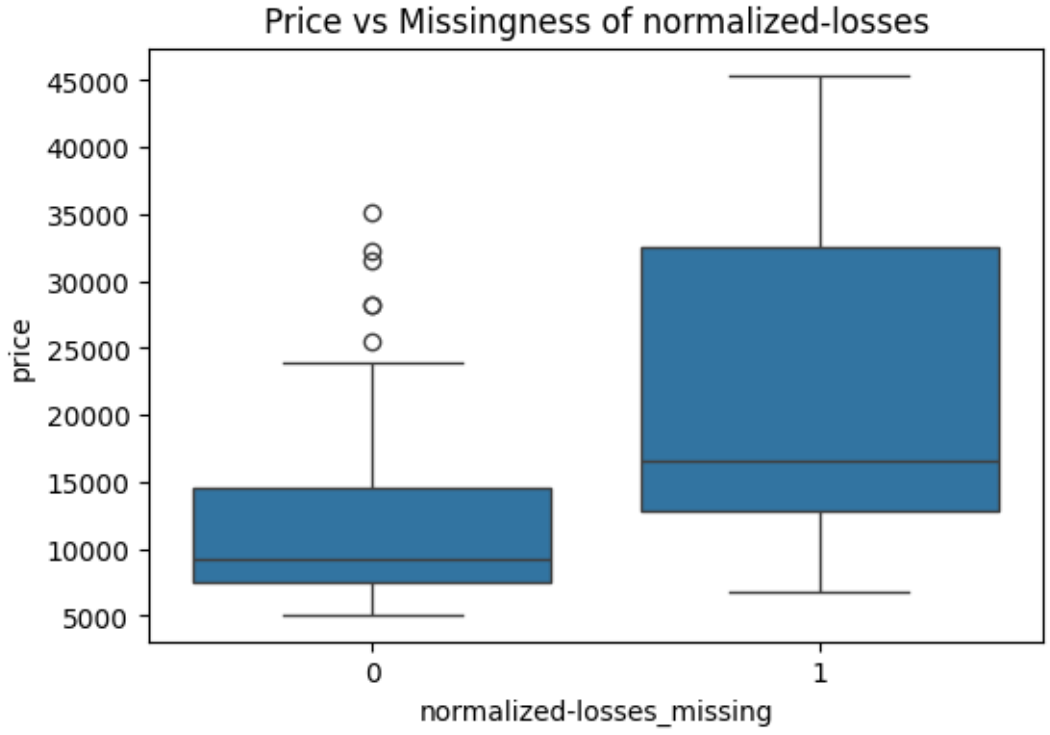
    plt.figure(figsize=(6, 4))

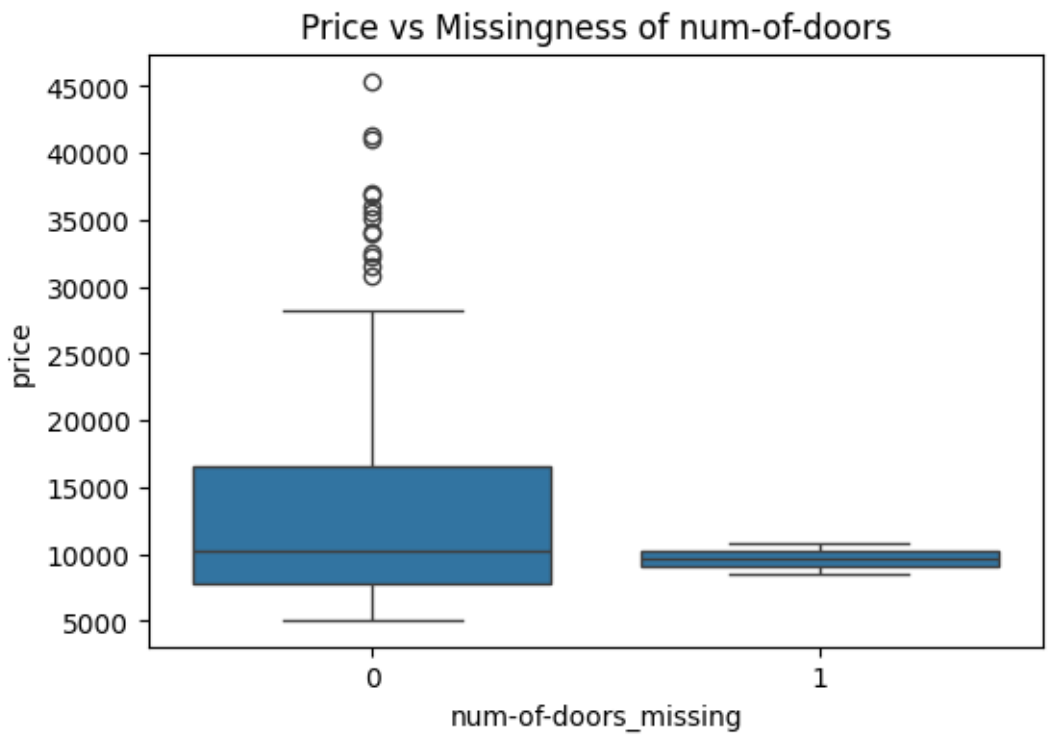
    sns.boxplot(x=missing_col_name, y='price', data=df)

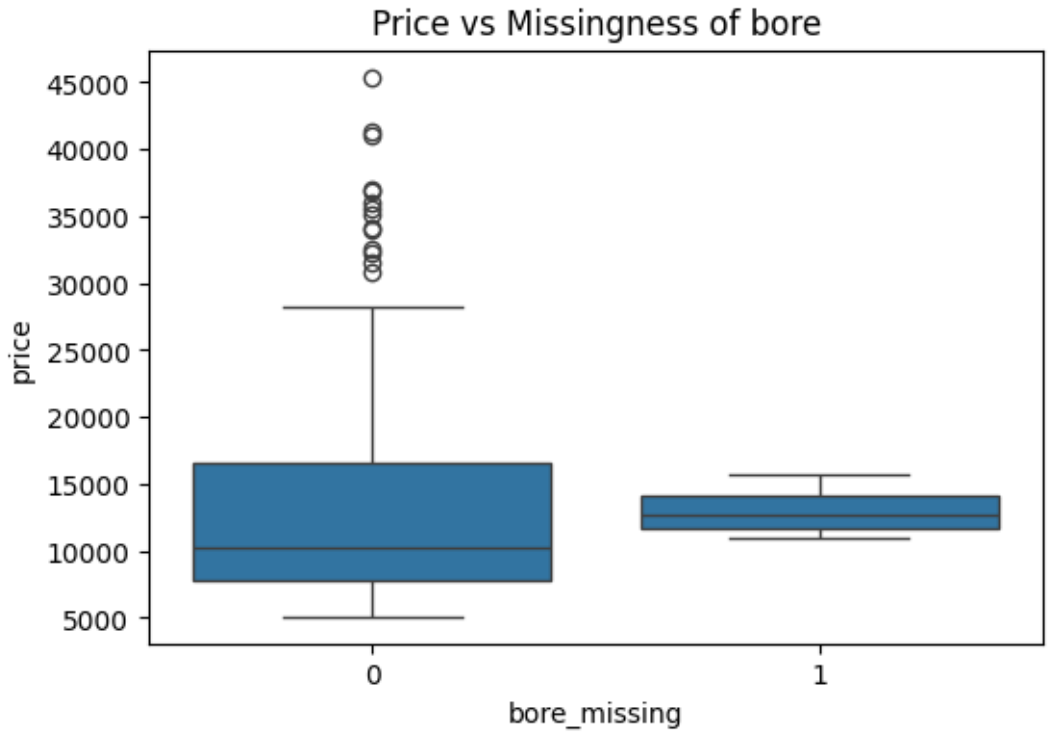
    plt.title(f'Price vs Missingness of {col}')

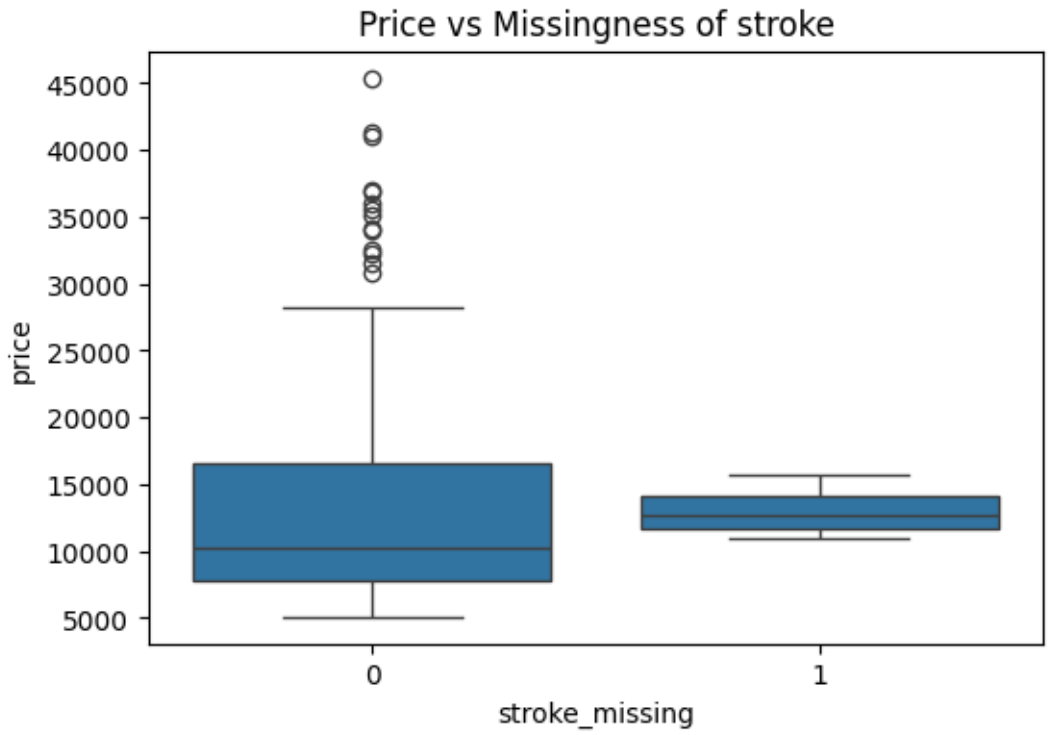
    plt.show()

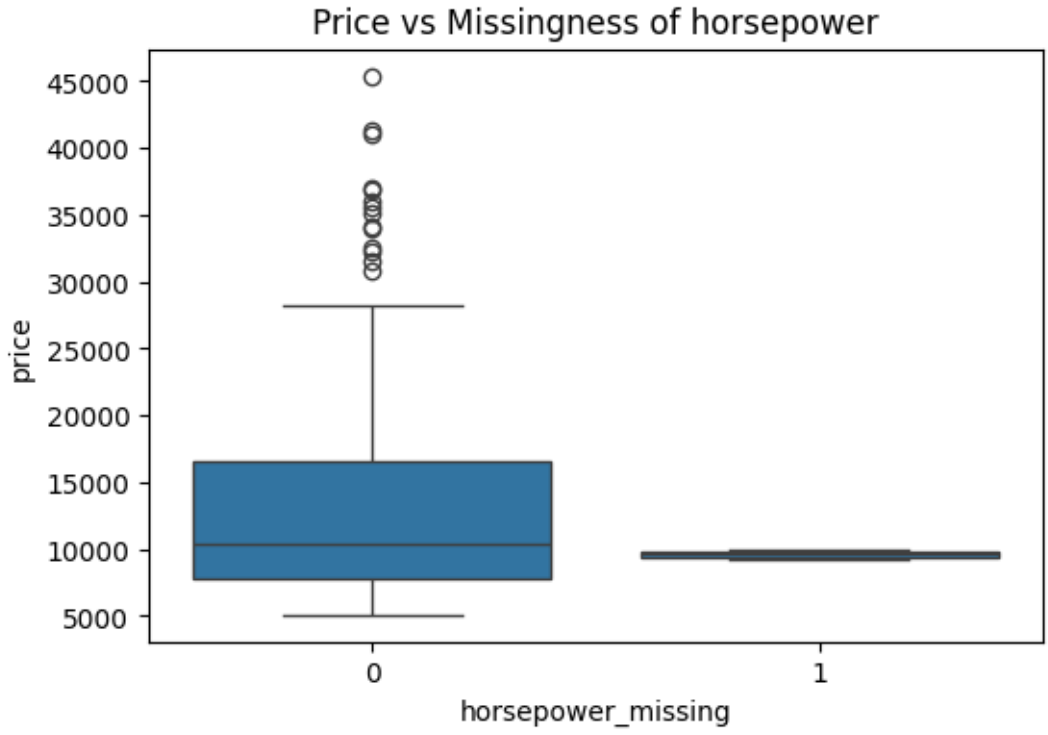
Output: The output will show the missingness of the variables with the price values. This will help us to choose the imputation method of the missing values in the independent variables.




Price vs Missingness of num-of-doors
price
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
1
num-of-doors_missing


Price vs Missingness of bore
price
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
1
bore_missing





Only these five independent variables have missing values in the dataset. Except Normalization losses, all the other missing data are MCAR type. So, we can choose the central tendency imputation method to impute those values. Also, for Normalized losses, we need to check the dataset further.

Step 10: Imputation of no. of doors:

For dodge makers, sedan types of cars the no. of doors is always four.

For Mazda makers, sedan types of cars the no. of doors is always four.

R Code: which(is.na(b2$`num-of-doors`))

b2[27,6]<-"four"

b2[61,6]<-"four"

View(b2)

Step 11: Imputation of Bore:

As the value is numeric and the data is almost evenly distributed so we can use the mean imputation method here.

R Code: b2$bore[which(is.na(b2$bore))]=mean(b2$bore,na.rm = T)

View(b2)

Step 12: Imputation of Horsepower & Peak RPM:

Both the data missing is in Renault Maker car data, and the whole data set have only those two Cars of Renault Maker. Deleting the Renault Car maker data will not impact much as in both the cases the data is missing.

R Code: which(is.na(b2$horsepower))

b2<-b2[-c(127,128),]

View(b2)

Step 13: Imputation of Normalized Losses:

In the dataset, the missingness of the “Normalized Losses” are almost normally distributed. So, we can impute the missing values with mean imputation method.

R Code: boxplot(b2$`normalized-losses`) #To Check the data distribution pattern

hist(b2$`normalized-losses`) #To Check the data distribution pattern

b2$`normalized-losses`[which(is.na(b2$`normalized-losses`))]=mean(b2$`normalized-losses`,na.rm = T)

View(b2)

Now the dataset is completed. I have saved the dataset in name of “Import_New”.

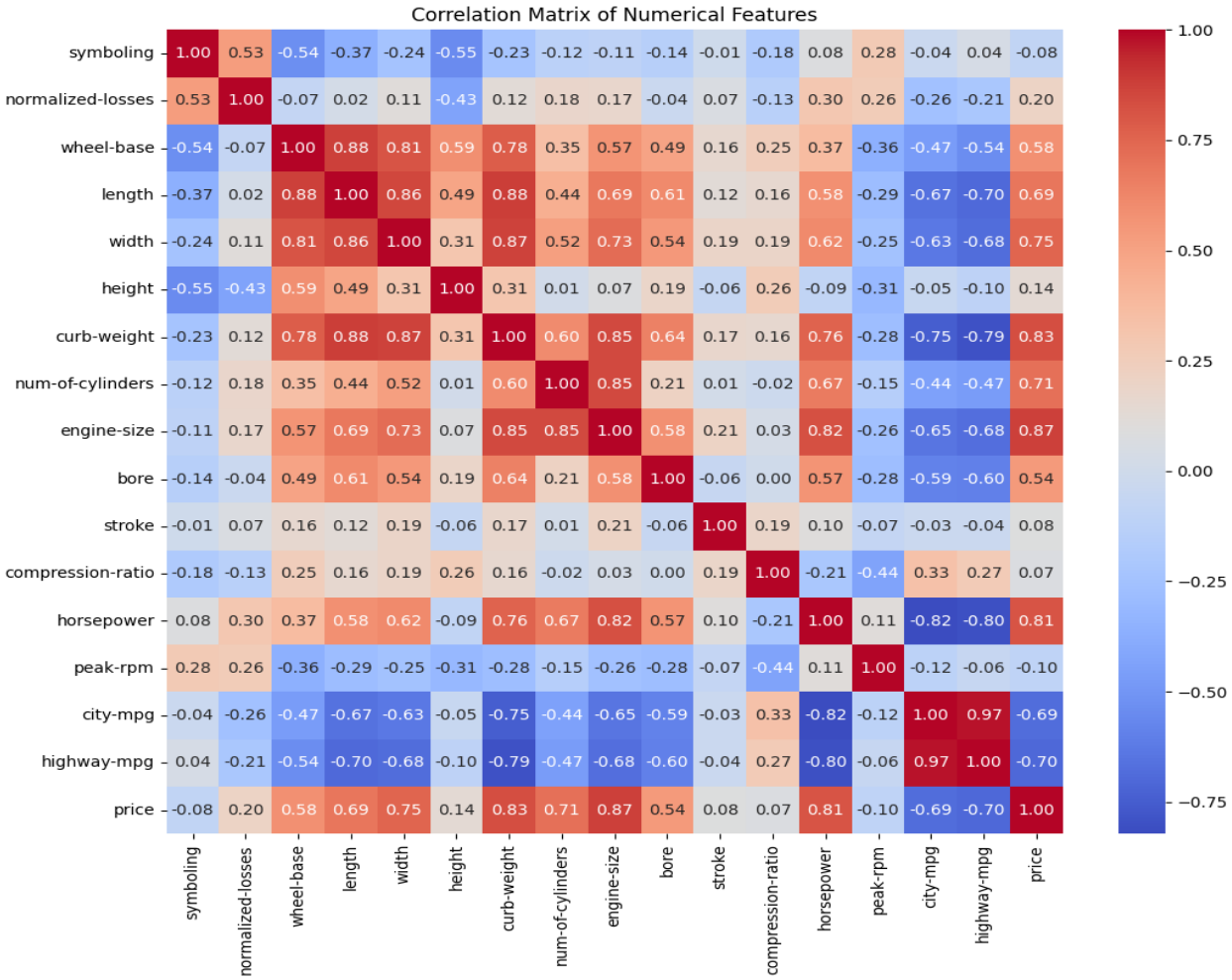
Now based on this preprocessed and cleaned dataset we have to run the linear regression model.

Step 14: Checking the correlation heatmap matrix to identify the relationship between the variables.

Python Code:

```
import pandas as pd

df = pd.read_csv('/content/Import_New.csv')
display(df.head())
display(df.info())

numeric_df = df.select_dtypes(include=['number'])
correlation_matrix = numeric_df.corr()
display(correlation_matrix)

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```

Output:



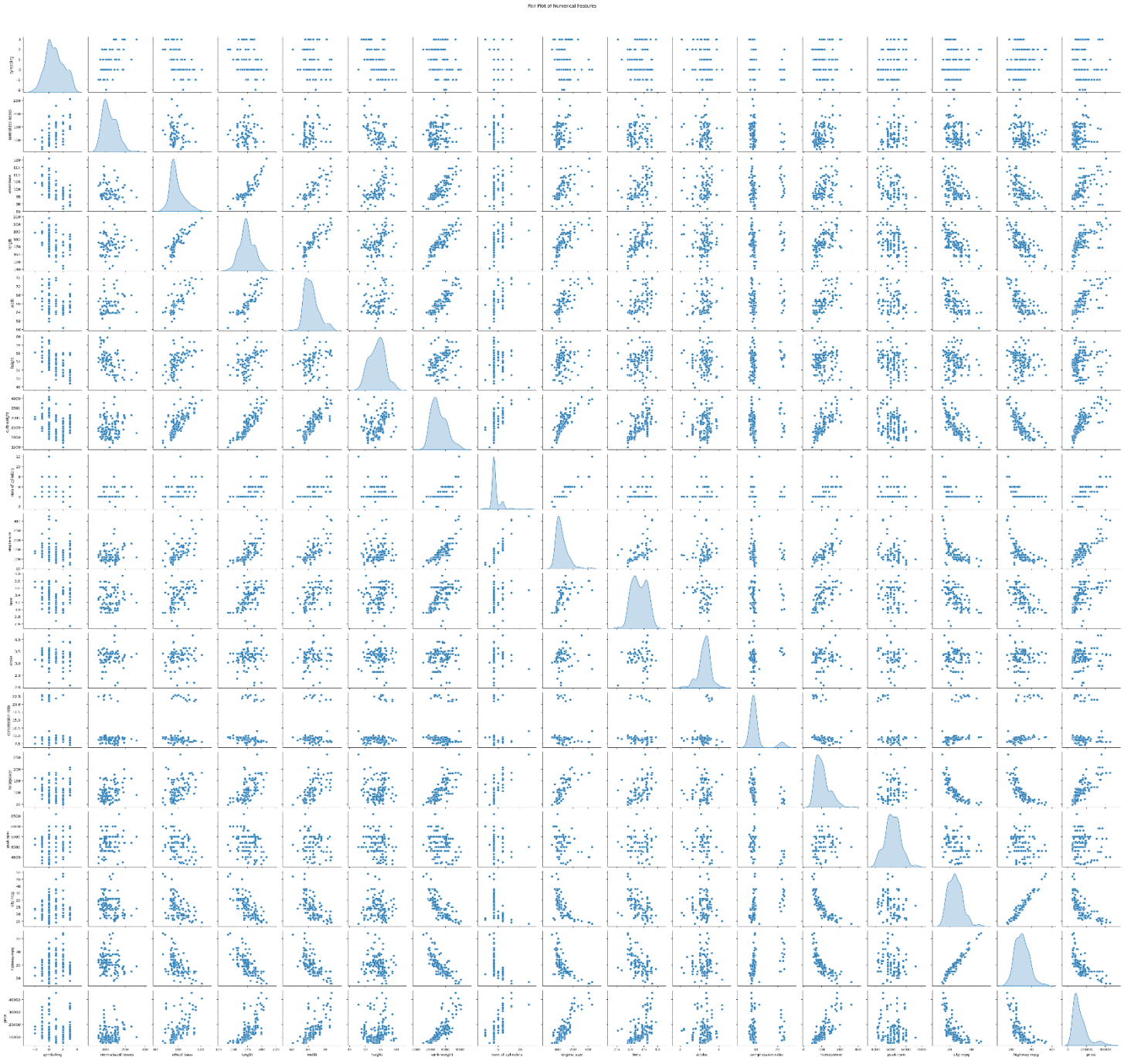
From the Correlation Matrix it is clear that the target variable has high correlation with: Length, Width, curb-weight, num-of-cylinders, engine size, horsepower, highway-mpg, city-mpg.

Step 15: Checking the pair plots:

Python Code: sns.pairplot(numeric_df, diag_kind='kde')

plt.suptitle('Pair Plot of Numerical Features', y=1.02)

plt.show()



Step 16: Dividing the dataset into train and test dataset:

R Code: #Split the dataset into train & test data

```
set.seed(231)
data=sample(2,nrow(b2),replace=T, prob = c(0.8,0.2))
data
train1=b2[data==1,]
test1=b2[data==2,]
train1
test1
```

Step 17: Final Model Building:

Let's just consider the variables which have high correlation with Price. They are:

Length, Width, curb-weight, num-of-cylinders, engine size, horsepower, highway-mpg, city-mpg.

Now, Length and Width have strong correlation between them. So, we can take any of them in the model.

No. of cylinder doesn't have high impact, Also, highway_mpg and city_mpg.

Final Model will Consist: Engine Size, Horsepower, Width/Length.

R Code: model_Price <- lm(price ~ width + horsepower +`engine-size`, data = train1)

summary(model_Price)

```
> #Final Model
> model_Price <- lm(price ~  width + horsepower +`engine-size`, data = train1)
> summary(model_Price)

Call:
lm(formula = price ~ width + horsepower + `engine-size`, data = train1)

Residuals:
    Min      1Q  Median      3Q     Max
-8613.1 -1882.6    52.3  1578.3 14284.7

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   -68126.58   11420.18  -5.965 1.53e-08 ***
width            976.14     186.13   5.244 4.93e-07 ***
horsepower        55.66      12.47   4.464 1.52e-05 ***
`engine-size`     89.70      12.36   7.257 1.65e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3497 on 159 degrees of freedom
Multiple R-squared:  0.8254,    Adjusted R-squared:  0.8221
F-statistic: 250.6 on 3 and 159 DF,  p-value: < 2.2e-16
```

From the output we can check that,
Dependent Variable: Price
Independent Variable: Width, Horse Power, Engine Size

Residuals:	Min	1Q	Median	3Q	Max
	-8613.1	-1882.6	52.3	1578.3	14284.7

Multiple R-squared: 0.8254
Adjusted R-squared: 0.8221
Model explains 82% of price variation.

Step 18: Check on the Test1 Dataset:

MSE Value	RMSE Value	MAE Value	R^2 Test Value
10333605	3214.592	2160.373	0.717997

R Code: predictions <- predict(model_Price, newdata = test1)

```
mse <- mean((test1$price - predictions)^2)
mse

rmse <- sqrt(mse)
rmse

mae <- mean(abs(test1$price - predictions))
mae

SSE <- sum((test1$price - predictions)^2)
SST <- sum((test1$price - mean(test1$price))^2)
R2_test <- 1 - SSE/SST
R2_test
```

The Model can predict 72% values of the test dataset accurately.

Step: 19: Visualize Actual Datapoints vs Predicted Datapoints: